

Documentação Completa do AETHER

Plataforma de Inteligência Climática Baseada no ODS 13 (Ação contra a Mudança Global do Clima)

O AETHER é uma API de inteligência de backend (construída com FastAPI) desenhada para processar, analisar e alertar autonomamente sobre riscos climáticos em qualquer região do globo.

Abaixo, explicamos detalhadamente a arquitetura do projeto e a responsabilidade de cada arquivo.

1. O Ponto de Entrada (`app/main.py`)

O `main.py` é o "coração" onde o servidor liga.

- **O que faz:** Inicia a aplicação FastAPI, registra as rotas (URLs) que o usuário pode acessar e gerencia o ciclo de vida do servidor (`lifespan`).
- **Por que é assim:** O `lifespan` garante que o banco de dados seja inicializado e que o **Motor de Monitoramento Autônomo (APScheduler)** seja ligado assim que o servidor subir, e encerrado corretamente quando o servidor desligar.

2. A Comunicação com a Rede

Para entregar previsões, o sistema precisa conversar com a internet.

`app/services/geocoder.py`

- **O que faz:** Quando o usuário digita "Recife, PE", o computador não entende texto, só coordenadas. Este arquivo converte o nome da cidade em Latitude e Longitude conectando-se ao serviço gratuito Nominatim (OpenStreetMap).
- **Por que é assim:** Usa `httpx.AsyncClient` e um padrão chamado **Connection Pooling**. Isso significa que ele deixa a porta de conexão com a internet aberta para não perder tempo conectando de novo em futuras pesquisas. Além disso, usa `@alru_cache` para "decorar" (salvar na memória) cidades já pesquisadas.

`app/data_pipeline/open_meteo.py`

- **O que faz:** Puxa os dados climáticos reais (chuva, vento, calor, pressão) baseados na latitude e longitude. Ele puxa o "agora" e uma matriz de previsão para os próximos 7 dias.
 - **Por que é assim:** Também usa Pooling e Cache. A API da Open-Meteo é gratuita e não exige chave, ideal para esse projeto.
-

3. Os Cérebros de Inteligência (Módulos AI)

`app/ai/climate_analyzer.py`

- **O que faz:** É a mente dedutiva do AETHER. Ele pega os dados da Open-Meteo e passa por 6 heurísticas matemáticas diferentes: **Inundação, Seca, Onda de Calor, Tempestade, Deslizamento e Incêndio Florestal**. Ele cruza vento com calor para saber o risco de fogo, ou chuva com umidade do solo para saber o risco de enchente.
- **Por que é assim:** Centraliza as regras de negócio de clima. Ele devolve uma lista de probabilidades e severidades (BAIXO, MODERADO, ALTO, CRÍTICO).

app/ai/trend_analysis.py

- **O que faz:** É a mente preditiva. Pega os arrays de previsão dos próximos 7 dias e desenha mentalmente uma curva de tendência.
- **Por que é assim:** Em vez de só falar o risco de hoje, ele gera recomendações textuais dinâmicas (ex: "*A chuva vai piorar na quinta-feira*"), trazendo um nível de inteligência sem depender de APIs pagas como o ChatGPT.

app/geospatial/processor.py

- **O que faz:** Usa a biblioteca geopandas e shapely para criar formas geográficas matemáticas (WKT).
- **Por que é assim:** Banco de dados espaciais (PostGIS) exigem polígonos matemáticos para saber a área afetada. Ele cria um círculo (buffer) em torno da cidade pesquisada.

4. O Sistema Nervoso (Banco de Dados e Rotas)

app/core/database.py

- **O que faz:** Configura o banco de dados `aether.db`. Possui as tabelas: `RiskAnalysis` (histórico de consultas), `Alert` (alertas críticos ativos) e `RegionSubscription` (cidades monitoradas).
- **Por que é assim:** Usa `aiosqlite` (SQLite Assíncrono). Bancos normais travam o servidor enquanto salvam arquivos. O assíncrono manda salvar no background e continua atendendo outros clientes na mesma fração de segundo.

app/api/v1_router.py

- **O que faz:** É o maestro. Define todas as rotas (endpoints). Ele orquestra todos os arquivos acima:
 1. Chama o Geocoder.
 2. Chama a Open-Meteo.
 3. Manda os dados para o `ClimateAnalyzer` e `TrendAnalyzer`.
 4. Salva a consulta no banco de dados.
 5. Aciona o `AlertEngine`.
 6. Devolve o JSON pronto para o site.

app/models/schemas.py

- **O que faz:** É o "molde" (Pydantic). Define exatamente o formato em que a resposta JSON sairá do backend e validará o que o Frontend mandar.
-

5. O Monitoramento Invisível

`app/worker/scheduler.py`

- **O que faz:** É um trabalhador de fundo (Background Worker). De tempos em tempos (ex: 2 em 2 horas), ele abre o banco de dados, olha a tabela `RegionSubscription`, puxa as cidades e manda analisar o clima delas sozinho, sem nenhum usuário humano ter pedido.
- **Por que é assim:** Transforma o AETHER de um sistema "reativo" para uma plataforma "autônoma". Se um temporal se formar de madrugada, o Scheduler detectará e lançará um alerta no banco de dados automaticamente.

`app/services/alert_engine.py`

- **O que faz:** É acionado quando o `ClimateAnalyzer` percebe que uma métrica atingiu nível **ALTO** ou **CRÍTICO**. Ele escreve o alerta de perigo público no banco de dados, marcando `is_active=True`.

Resumo da Ópera: O projeto AETHER é uma malha de microserviços internos. Ele é extremamente otimizado por via de requisições assíncronas e LRU Cache, impedindo gargalos. Suas heurísticas cruzam dados atmosféricos simulando o trabalho de um meteorologista, enquadrando todas as análises estritamente nas metas do **ODS 13 da ONU**.